Nginx et Let's Encrypt sur un serveur cloud

BTS SIO 2e année SISR

Projet fil rouge · mardi 3 novembre 2026 · Blocs 2 et 3

Service web exposé sur Internet

Objectifs

- Administrer à distance un serveur Debian réellement exposé sur Internet.
- Publier un site avec Nginx et vérifier le lien nom → adresse.
- Obtenir un certificat d'une autorité publique par le protocole ACME, puis passer en HTTPS.
- Automatiser le renouvellement et mesurer ce que l'exposition publique implique.

Prérequis

- Votre paire de clés SSH, générée lundi ; la clé publique a été transmise à l'enseignant.
- DNS (enregistrement A), requête HTTP, codes 200, 301 et 404.
- Certificat X.509, autorité de certification, clé privée (Bloc 3, P01 et P06).

Sommaire

Étape	Travail
1	Prendre possession du serveur
2	Nginx en HTTP
3	Un certificat Let's Encrypt
4	HTTPS, redirection et preuves
5	Renouvellement et exposition publique
6	Livrable et restitution

Votre mission

Chacun dispose d'un **serveur Debian chez un hébergeur cloud** (OVHcloud, instance d2-2 : 1 vCPU, 2 Go de mémoire), avec une adresse IPv4 publique et un nom DNS de la forme `sisrN.tp.siovision.fr`, où **N est le numéro donné par l'enseignant**. Remplacez `N` partout dans ce document.

Contrairement au laboratoire, ce serveur est joignable par n'importe qui sur Internet. Vous y installez Nginx, puis vous obtenez un certificat d'une autorité **publique**, Let's Encrypt : tout navigateur fera confiance à votre site, sans manipulation côté client.

Travail individuel, un serveur par étudiant. Le serveur est détruit en fin de TP : rien de ce que vous y placez ne doit être personnel ni réutilisé ailleurs. Complétez un tableau **commande / attendu / observé / conclusion** au fil des étapes.

1. Prendre possession du serveur

Depuis votre poste, avec la clé générée lundi :

```
ssh etudiant@sisrN.tp.siovision.fr
```

À la première connexion, SSH affiche l'empreinte de la clé du **serveur** et demande confirmation. Acceptez, puis vérifiez où vous êtes :

```
hostname -f; ip -br address; ip route
sudo ss -ltnp
getent ahostsv4 sisrN.tp.siovision.fr
```

Question 1. Comparez l'adresse IPv4 de `ip -br address` et la réponse de `getent`. Qu'est-ce qui garantit que le nom mène bien à votre serveur ? Que vérifie SSH quand il vous demande d'accepter l'empreinte, et que se passerait-il si vous l'acceptiez sans la contrôler sur un serveur de production ?

Question 2. Quels services écoutent, et sur quelles adresses ? Lesquels sont joignables depuis Internet ?

2. Nginx en HTTP

```
sudo apt update
sudo apt install -y nginx certbot
sudo mkdir -p /var/www/sisrN
sudo nano /var/www/sisrN/index.html
```

Rédigez une page courte : un titre et la mention « Serveur sisrN, 2SISR », sans donnée personnelle. Créez le site `/etc/nginx/sites-available/sisrN` :

```
server {
    listen 80;
    server_name sisrN.tp.siovision.fr;
    root /var/www/sisrN;

    access_log /var/log/nginx/sisrN-access.log;
    error_log  /var/log/nginx/sisrN-error.log;
}
```

Activez-le, **validez avant de recharger** :

```
sudo ln -s /etc/nginx/sites-available/sisrN /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

Testez depuis **votre poste**, puis ouvrez l'adresse dans un navigateur. Sous Windows, tapez `curl.exe` (dans PowerShell, `curl` désigne une autre commande).

```
curl -i http://sisrN.tp.siovision.fr/
curl -i http://sisrN.tp.siovision.fr/absent
```

Attendu : `200 OK` avec votre page, puis `404 Not Found`. Le navigateur signale le site comme « non sécurisé ».

Question 3. Le site Debian par défaut est toujours actif. Que renvoie `curl -i http://ADRESSE_IP/` ? Pourquoi ?

3. Un certificat Let's Encrypt

Let's Encrypt ne vous connaît pas. Avant de signer, il doit vérifier que **vous contrôlez le nom** demandé. Le protocole ACME lui sert à le faire : avec le défi **HTTP-01**, certbot dépose un fichier dans `/.well-known/acme-challenge/` de votre site, puis les serveurs de Let's Encrypt viennent le lire **par Internet, sur le port 80**.

```
sudo certbot certonly --webroot -w /var/www/sisrN \
  -d sisrN.tp.siovision.fr \
  --agree-tos --register-unsafely-without-email
```

Pendant l'exécution, surveillez le journal dans un second terminal SSH :

```
sudo tail -f /var/log/nginx/sisrN-access.log
```

En cas d'échec, lisez le message, diagnostiquez, puis relancez. Let's Encrypt bloque temporairement un nom après plusieurs échecs rapprochés. Journal détaillé : `/var/log/letsencrypt/letsencrypt.log`.

Inspectez le résultat :

```
sudo ls -l /etc/letsencrypt/live/sisrN.tp.siovision.fr/
sudo openssl x509 -in /etc/letsencrypt/live/sisrN.tp.siovision.fr/cert.pem \
  -noout -subject -issuer -dates -ext subjectAltName
```

Question 4. Dans le journal d'accès, retrouvez les requêtes de Let's Encrypt : quelle URL, combien de requêtes, depuis combien d'adresses différentes ? Pourquoi plusieurs points de vérification ?

Question 5. Où se trouve la clé privée du certificat ? A-t-elle été envoyée à Let's Encrypt ? En P06, vous transmettiez une CSR à l'autorité du laboratoire : quelle étape remplace ici votre copier-coller ?

Question 6. Quelle est la durée de validité du certificat ? Pourquoi une autorité publique choisit-elle une durée aussi courte ?

4. HTTPS, redirection et preuves

Remplacez le contenu de `/etc/nginx/sites-available/sisrN` par deux blocs : le port 80 ne fait plus que rediriger.

```
server {
    listen 80;
    server_name sisrN.tp.siovision.fr;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name sisrN.tp.siovision.fr;
    root /var/www/sisrN;

    ssl_certificate     /etc/letsencrypt/live/sisrN.tp.siovision.fr/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/sisrN.tp.siovision.fr/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;

    access_log /var/log/nginx/sisrN-access.log;
    error_log  /var/log/nginx/sisrN-error.log;
}
```

```
sudo nginx -t && sudo systemctl reload nginx
sudo ss -ltnp | grep nginx
```

Depuis votre poste, dans cet ordre :

```
curl -I http://sisrN.tp.siovision.fr/
curl -i https://sisrN.tp.siovision.fr/
openssl s_client -connect sisrN.tp.siovision.fr:443 \
  -servername sisrN.tp.siovision.fr < /dev/null 2>&1 \
  | grep -E 'subject=|issuer=|Verify return|Protocol|Cipher'
```

Attendu : `301` vers `https://…`, puis `200 OK` **sans** `--cacert`, puis `Verify return code: 0 (ok)`. Si `openssl` manque sur votre poste, lancez cette commande depuis le serveur : elle passe aussi par le nom public. Ouvrez le site dans un navigateur et affichez le certificat (cadenas).

Question 7. En P06, la même requête `curl` échouait sans `--cacert`. Pourquoi réussit-elle ici sans aucune option ? Où se trouve le certificat de l'autorité qui permet cette vérification ?

Question 8. Nginx utilise `fullchain.pem` et non `cert.pem`. Que contient en plus `fullchain.pem` ? Que se passerait-il chez certains clients avec `cert.pem` seul ?

Repères pour interpréter une erreur :

Observation	Cause probable	Premier contrôle
certbot : `Timeout during connect`	Port 80 injoignable depuis Internet	`ss -ltnp`, Nginx actif, pare-feu
certbot : `NXDOMAIN` ou mauvaise adresse	Nom non publié ou erroné	`getent ahostsv4 sisrN.tp.siovision.fr`
certbot : `404` sur `acme-challenge`	`-w` différent de la racine du site	`root` dans Nginx, chemin `-w`
`Connection refused` sur 443	Nginx n'écoute pas en HTTPS	`nginx -t`, `listen 443 ssl`
`nginx -t` : `cannot load certificate`	Chemin faux ou certificat absent	`sudo ls /etc/letsencrypt/live/`

5. Renouvellement et exposition publique

Un certificat court n'a d'intérêt que si son renouvellement est **automatique** :

```
systemctl list-timers | grep certbot
sudo certbot renew --dry-run
```

Après un renouvellement, Nginx doit recharger le nouveau certificat. Ajoutez un crochet de déploiement, exécuté par certbot à chaque renouvellement réussi :

```
printf '#!/bin/sh\nsystemctl reload nginx\n' \
  | sudo tee /etc/letsencrypt/renewal-hooks/deploy/reload-nginx
sudo chmod 755 /etc/letsencrypt/renewal-hooks/deploy/reload-nginx
```

Mesurez maintenant l'exposition de votre serveur :

- cherchez votre nom sur crt.sh : `sisrN.tp.siovision.fr` ;
- relisez les journaux :

```
sudo awk '{print $1, $7}' /var/log/nginx/access.log /var/log/nginx/sisrN-access.log \
  | sort | uniq -c | sort -rn | head -20
sudo journalctl -u ssh --since today | grep -cE 'Invalid user|Failed'
```

Question 9. Que fait le temporisateur `certbot.timer`, et à quelle fréquence ? Pourquoi le crochet de rechargement est-il nécessaire ?

Question 10. Votre nom apparaît sur crt.sh quelques minutes après l'émission. Pourquoi les autorités publient-elles chaque certificat ? Quelle conséquence pour un nom de serveur que l'on voudrait garder discret ?

Question 11. Quelles requêtes ne viennent ni de vous ni de Let's Encrypt ? Que cherchent-elles ? Proposez deux mesures pour réduire l'exposition du serveur (SSH, services, pare-feu).

Pour les rapides : comparez avec la méthode automatique. Sauvegardez votre configuration, lancez `sudo certbot --nginx -d sisrN.tp.siovision.fr`, puis `diff` entre l'ancienne et la nouvelle : qu'a modifié certbot ? Ajoutez ensuite l'en-tête `Strict-Transport-Security` avec une durée courte et expliquez son rôle.

6. Livrable et restitution

Rendez un dossier court : schéma (poste ⟶ Internet ⟶ serveur Nginx 80/443, Let's Encrypt qui vient vérifier sur le port 80) ; configuration Nginx finale ; tableau **commande / attendu / observé / conclusion** ; extraits de

journaux reliés à vos requêtes ; réponses aux questions. Aucune clé privée dans le dossier : ni `privkey.pem`, ni votre clé SSH.

Chaque étudiant rédige une conclusion de **6 à 8 lignes** : en quoi la confiance accordée à votre site diffère de celle du laboratoire en P06 (qui vérifie quoi, et comment), et ce que l'exposition sur Internet a changé.

Réussite : `301` de HTTP vers HTTPS, `200 OK` en HTTPS sans option depuis votre poste, certificat émis par Let's Encrypt pour votre nom, `renew --dry-run` réussi, explication sans notes de la question 7.

Repères

ACME automatise la preuve de contrôle d'un nom et la délivrance du certificat ; la clé privée reste sur le serveur. Références : Let's Encrypt, fonctionnement, types de défis, certbot, documentation, Nginx, HTTPS.

L'IA peut expliquer un message d'erreur après une première analyse personnelle. Vérifiez toute commande avant de l'exécuter, ne transmettez jamais de clé privée ni le contenu de `/etc/letsencrypt/`, et rédigez votre conclusion vous-même.